Zhejiang University – University of Illinois at Urbana-Champaign Joint Institute

Quiz #4

CS 225 Data Structures

Spring 2026

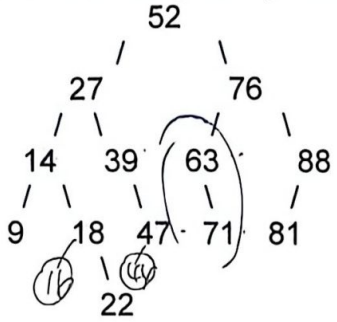
7:00-8:00pm, Thursday, April 9

Name	Hanbin Zhang
ZJU ID	
Lab TA	

- This is a **closed book** and **closed notes** exam. No electronic aids are allowed, either.
- You should have 4 problems total on 9 pages. You must write on the exam.
- Unless otherwise stated in a problem, assume the best possible design of a particular implementation is being used.
- Unless the problem specifically says otherwise, (1) assume the code compiles, and thus any compiler error is an exam typo (though hopefully there are not any typos), and (2) assume you are NOT allowed to write any helper methods to help solve the problem, nor are you allowed to use additional arrays, lists, or other collection data structures unless we have said you can.
- Please put your name at the top of each page.
- Please fill in your **Lab Section** or your **Lab TA's name** to help us organize and grade your papers.
- Don't panic and good luck!

Problem 1	_____/10
Problem 2	_____/10
Problem 3	_____/10
Problem 4	_____/10
Total	_____/40

Problem 1 (10 points): Tree Basics and Traversal
Consider the following binary tree:



Answer the following questions.

d 1. Suppose this tree is implemented using the usual node-based binary tree structure, where each node stores exactly two child pointers. How many **NULL** pointers are there in this tree?
a) 11
b) 12
c) 13
d) 14
e) 15

d 2. Which of the following statements is **correct**?
a) The tree is full but not complete
b) The tree is complete but not full
c) The tree is both full and complete
d) The tree is neither full nor complete
e) The tree is not a binary tree

e 3. What is the **inorder traversal** of this tree?
a) 9, 14, 18, 22, 27, 39, 47, 52, 63, 71, 76, 81, 88
b) 9, 14, 18, 22, 27, 47, 39, 52, 63, 71, 76, 81, 88
c) 9, 14, 22, 18, 27, 39, 47, 52, 63, 71, 76, 81, 88
d) 14, 9, 18, 22, 27, 39, 47, 52, 63, 76, 71, 81, 88
e) 9, 14, 18, 22, 27, 39, 47, 52, 71, 63, 76, 81, 88

4. Assume this tree is a binary search tree.
(1) If a new key **44** is inserted, it becomes the left (left/right) child of node 47.
(2) If a new key **16** is inserted, it becomes the left (left/right) child of node 18.

5. Assume a binary tree node is defined as follows:

```
struct TreeNode {
    int key;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int k) : key(k), left(nullptr), right(nullptr) { }
};
```

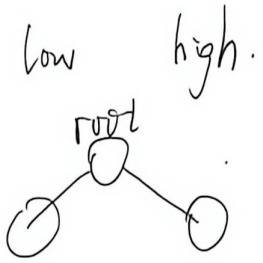
Complete the following function so that it **prints all keys** in the tree in **postorder** traversal. You may use print() function.
You **must use recursion** (i.e., the function must call itself). Solutions that do not use recursion will receive no credit.

```
void postorder(TreeNode* root) {
    if (root == nullptr) {
        return;
    }
    _____
    _____
    _____
    _____
    _____
}
```

Problem 2 (10 points): Binary Search Tree

```
struct Node {
    int key;
    Node* left;
    Node* right;
};
```

Assume all keys are distinct.



(a) Recursive BST validity check

Complete the correct recursive helper below.

```
bool isBST(Node* root, int low, int high) {
    if (root == nullptr) return true;

    if (root->key <= low || root->key >= high)
        return false;

    return isBST(root->left, ____[1]____, ____[2]____) &&
           isBST(root->right, ____[3]____, ____[4]____);
}
```

[1] low
[2] root->key
[3] root->key
[4] high

(b) BST find

```
Node* find(Node* root, int k) {
    if (root == nullptr || root->key == k)
        return ____[5]____

    if (k < root->key)
        return find(root->left, k);
    else
        return find(____[6]____, k);
}
```

[5] root
[6] root->right

Name:

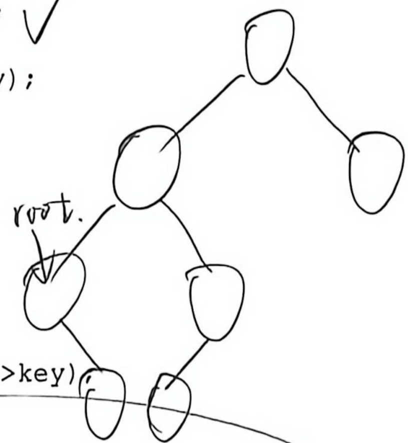
(c) BST delete

```
Node* maxNode(Node* root) {
    while (root->right != nullptr)
        root = root->right;
    return root;
}

Node* deleteNode(Node* root, int key) {
    if (root == nullptr) return nullptr;

    if (key < root->key) {
        root->left = deleteNode(root->left, key);
    } else if (key > root->key) {
        root->right = deleteNode(root->right, key);
    } else {
        if (root->left == nullptr)
            return ____[7]____;
        if (root->right == nullptr)
            return ____[8]____;

        Node* pred = maxNode(_root->left_);
        root->key = ____[10]pred->key
        root->left = deleteNode(root->left, pred->key);
    }
    return root;
}
```



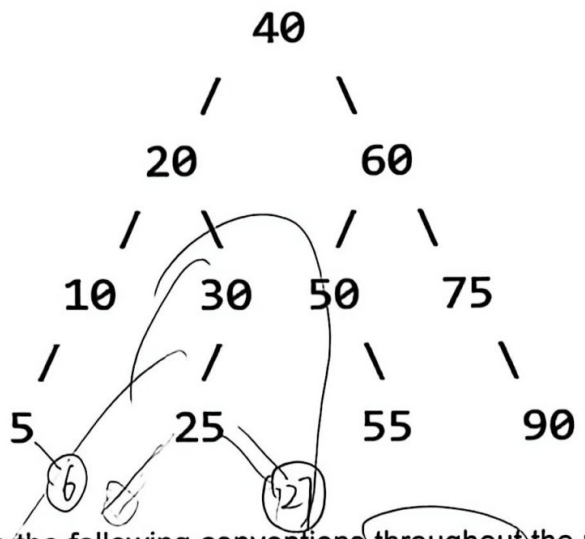
错题，没有 delete

不知道哪个像画出的 delete 函数没有 delete .

standard :

Problem 3 (10 points): AVL

Consider the following AVL tree T, whose keys are all distinct integers.



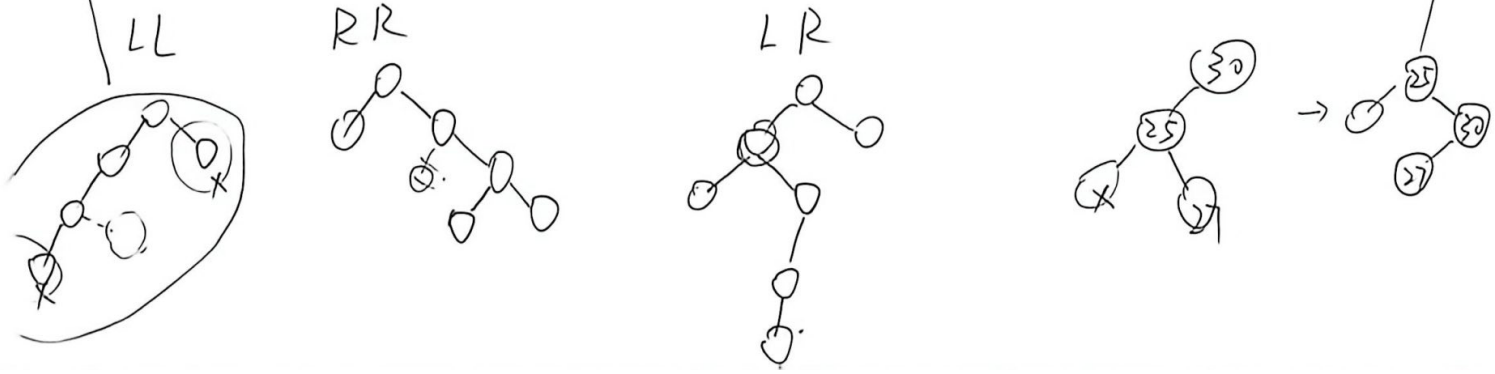
Assume the following conventions throughout the problem:

- The height of an empty subtree is −1.
- The height of a leaf is 0.
- Unless explicitly stated otherwise, each subproblem starts from the original tree shown below.

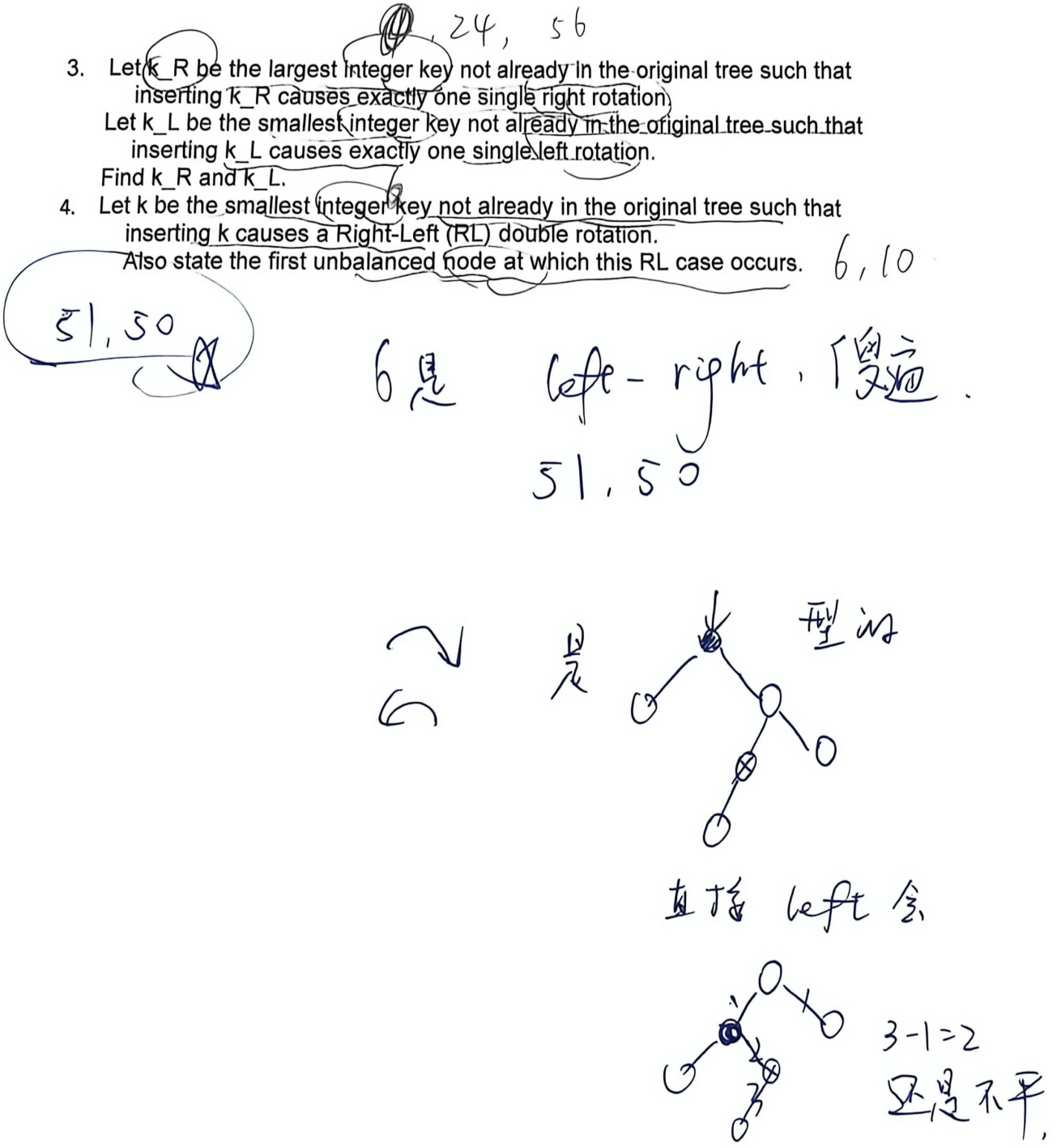
Let h denote the height of the tree and n denote the number of nodes.

1. If key 26 is searched for in the tree using the BST/AVL `find` procedure:
   - write the sequence of keys compared, in order;
   - give the worst-case running time of `find` in terms of h;
   - give the worst-case running time of `find` in terms of n.

2. If key 27 is inserted into the tree:
   - what is the first node on the path back to the root that becomes unbalanced?
   - what type of AVL imbalance occurs?
   - Please write the whole tree after insertion.



3. Let k_R be the largest integer key not already in the original tree such that inserting k_R causes exactly one single right rotation.
   Let k_L be the smallest integer key not already in the original tree such that inserting k_L causes exactly one single left rotation.
   Find k_R and k_L.
4. Let k be the smallest integer key not already in the original tree such that inserting k causes a Right-Left (RL) double rotation.
   Also state the first unbalanced node at which this RL case occurs.



Name:

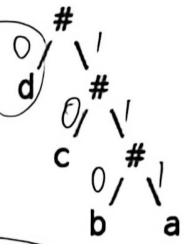
Problem 4 (10 points): Huffman Tree & Running Time

1. Huffman Tree

(1) Which of the following code sets cannot be a valid Huffman code for five distinct characters?

a) a: 0 b: 10 c: 110 d: 1110 e: 1111
b) a: 00 b: 01 c: 10 d: 110 e: 111
c) a: 0 b: 01 c: 10 d: 110 e: 111
d) All of the above can be valid Huffman codes.

(2) Given the following Huffman code: 0101101010111, and the following Huffman tree, what is the coded message (notice that the tree branches have not been denoted as 0 or 1)?



a)	c c d c c c d d	or	c d b c c d
b)	c d b c c d	or	d c b c c a
c)	d c b c c a	or	c c d c c c d d
d)	d c b c c a		
e)	c c d c c c d d		
f)	c d b c c d		

2. Running Time:

(1) You need a data structure that guarantees fastest search in the **worst case**. Which would you choose?

a) Unbalanced BST
b) AVL tree
c) Linked list
d) Unsorted array

(2) Consider a BST constructed by inserting **n** keys in **sorted order**. Answer the following:

A. What is the height of the tree? Write in **Big-O** notation.
B. What is the worst runtime of searching for an element? Write in **Big-O** notation.
C. Briefly explain why inserting the same set of keys into a BST in different order may result in different runtimes for search. No more than 15 words.

(3) What is the time complexity of deletion in an **AVL** tree of **n** elements? Besides, write down the steps and their costs respectively. *(Hint: There are 4 steps for deletion in an AVL tree, you need to correctly write the **Big-O** notation for all steps to get full credit.)*

AVL Tree Deletion Step	Time Complexity (Worst)
1. find tree	
2. determine whether	
3.	
4.	
Totally